

New Advances in B*

Pascal R. Anema

Department of Advanced Computing Sciences, Faculty of Science and Engineering, Maastricht University,
Maastricht, The Netherlands p.anema@student.maastrichtuniversity.nl

27 February 2025

Abstract. B* is a best-first tree search algorithm from 1979 in the race to beat humans at chess. It is still used for competitive endgame play of Scrabble, notably by MAVEN. In this report the range-based best-first search algorithm B* is re-tested on larger artificial game trees. For this purpose, more flexible artificial game trees are presented. It is shown that B*, even with a randomized strategy selection, gets increasingly more efficient relative to best-first search as tree size increases. A new enhancement for B* is introduced: Disprove-Best, which guides lower-level node selection to disprove the best node for faster proofs. Experiments show Disprove-Best B* requires fewer node expansions than B* and makes more progress on unsolved trees with limited resources, especially near the beginning of the search. We compare Disprove-Best B* with PB* and find similar performance, but note that the two algorithms are not mutually exclusive and could be combined in future research.

Keywords: B* Tree Search Algorithm · Artificial Game Trees · Adversarial Search · Artificial Intelligence · Game Playing

1 Introduction

1.1 Adversarial Game Playing Agents

For decades, game-playing has been an important part of AI development. In adversarial games like chess and Go, search algorithms like $\alpha\beta$ [7] and MCTS [5] have already made huge progress [4,12]. Many adversarial gameplay algorithms search through a so-called game tree to estimate the game-theoretical value of each available move.

However, these techniques stray from the original goal of choosing the best next move. They focus on refining a heuristic value for each possible move through extensive search [7], or through simulation [5], but do not provide guarantees of finding the best move. The lesser-known admissible best-first tree-search algorithm B^* [2] instead aims its search at proving the optimality of moves using a range-based evaluation function. It allows B^* to stop the search earlier than similar value estimation techniques, like $\alpha\beta$, while guaranteeing optimality. However, it requires an evaluation function which returns a lower and upper bound for the game-theoretical value.

Early research presented B^* [2] as a promising contender for playing the game of chess [8,9,3]. However, not much research was performed on B^* after DEEP BLUE [4] beat Garry Kasparov in 1997, which cemented $\alpha\beta$ [7] as the choice for chess programs since. Regardless, B^* remains competitive in the domain of endgame play in Scrabble as implemented in the Scrabble engine MAVEN [10].

1.2 The B^* Algorithm

The B^* [2] algorithm was conceived as a natural approximation of the protocols chess masters might apply to search for their next move. Unlike value-estimation search techniques, B^* is an admissible in-memory best-first algorithm. B^* is guaranteed to terminate and find the optimal solution given the heuristic evaluation function provides correct bounds. Finding such a heuristic can be difficult. However, it has been shown that B^* is fairly robust to occasional violations of this requirement in practice [2,11].

A related algorithm PB^* [8,9], developed as a generalization and improvement of B^* [2], instead uses a probability distribution as an evaluation function. This variation achieved higher efficiency than the B^* algorithm but has not seen much application in practice. The PB^* generalization improves the efficiency of the B^* algorithm, as Berliner described in 1996 [3]. However, it requires a more complicated evaluation function while also raising the complexity of the algorithm overall. Sheppard [11] considered using PB^* for the endgame search of MAVEN [10] as an alternative to B^* , but possibly stuck with the latter out of simplicity.

In general, no new research has focused on further improving the B^* [2] algorithm or continuing the work on PB^* [9] since 1996 [3]. This is surprising as the core principles of B^* are unique among game-tree search algorithms. B^* is a proving technique, like Proof-Number Search [1], that only terminates when the optimal move has been proven. Unlike Proof-Number Search, B^* does not require reaching a terminal position of the game to prove the optimality of a move. B^* can terminate earlier and with less overall effort than $\alpha\beta$ [7], MCTS [5], or Proof-Number Search, but requires a good heuristic and enough memory to achieve.

1.3 Problem Statement and Research Questions

The B^* [2] algorithm could be considered a relic of the race to beat humans in chess with computers. However, it appears to be much more than that. MAVEN [10] implements B^* almost exactly as originally described and achieves near-perfect play of Scrabble in the endgame. The original formulation of B^* is old but still good enough for state-of-the-art programs such as MAVEN and Quackle [6], while improvements are likely possible. For example, the uniform-distribution variant of PB^* [8] achieves better performance than B^* while utilizing the same range-based evaluation function. All research on the stand-alone performance of B^* has been conducted before 1997, on much smaller problems than could be tackled with today's hardware.

The **problem statement** of this report is:

How does B^ perform on larger problems and how can it be improved?*

The original B* paper [2] tested the B* algorithm on artificially generated game trees. These trees can easily be scaled up to test B*'s performance on larger problems. They could also be further improved to increase the variety of problems they can simulate. This leads to the first two **research questions**:

1. *How do existing implementations of the B* algorithm perform on larger artificial game trees?*
2. *How do the different parameters for tree generation affect their size and difficulty?*

Our problem statement also mentions a wish to improve the B* algorithm. We introduce a new variant of B* called Disprove-Best B* which aims to reach the performance enhancements of PB* [8] while keeping the complexity low. This leads to the final **research question**:

3. *How does Disprove-Best B* perform compared to existing B* variants?*

1.4 Outline

This report is structured as follows. First, Section 2 describes the concept of artificial game trees. Clarifying how they are generated according to existing research (2.1) and how they are expanded upon in this report (2.2). Second, Section 3 describes the B* [2] algorithm and some of its extensions in further detail. Subsequently, Section 4 introduces the novel Disprove-Best B* algorithm and explains its workings and requirements. Section 5 describes the setup and results of the experiments. Section 6 compares the results of our experiments with existing research. Finally, Section 7 uses the results of our experiments to answer the research questions and conclude this report with recommendations for future research.

2 Artificial Game Trees

Game-tree search algorithms, like $\alpha\beta$ [7] and B* [2], search for the best response to a game position by exploring a game tree, where nodes in that tree correspond to game positions. For single-player games, all nodes in the tree try to maximize some score received at the end of the game. Two-player games are often modelled as zero-sum games, where one player wants to maximize the score and the other wants to minimize it. If all players play optimally, such that any deviation from the strategy leads to a lower score for that player, the score received at the end of the game from a specific game position is called its game-theoretical value.

Many adversarial tree search algorithms use a heuristic to estimate the game-theoretical value at non-final game positions. The B* algorithm [2] uses two heuristics: pessimistic and optimistic evaluation. For any node n , the pessimistic value $Pes(n)$ is a non-decreasing function that is always less than or equal to the non-increasing optimistic value $Opt(n)$. The bound $[Pes(n), Opt(n)]$ should converge on and contain the game-theoretical value of n . This bound should get tighter as the uncertainty of the game-theoretical value decreases.

Finding good heuristics for B* [2] that fulfil these requirements can be difficult, depending on the domain. Because of this, we simulate searches on artificial game trees to evaluate the performance of algorithms. Performance on these trees tests the implementation-independent efficiency of an algorithm without relying on the quality of chosen heuristics. The following section describes how these trees are generated in this report.

2.1 Original Description

Unlike nodes of game trees, nodes in an artificial tree have no game position to store: only their name, parent, descendants, and heuristic values [2]. Moreover, artificial trees are generated during the search and can be infinitely expanded. Every node has a unique name determined by its parent and its position relative to its siblings. This name sets the seed of the pseudo-random number generator used to generate the node's heuristic values.

The trees we explore in this report have a constant branching factor b and rarely include transpositions (different sequences of moves leading to the same game position). Additionally, the root node of a tree is assigned a pessimistic value of 1 and an optimistic value of R where both heuristics can only take integer values. The B* paper [2] defined that the parent's bounds $[L, U]$

serve as bounds on the range of values that a node’s optimistic and pessimistic values may take. However, the paper did not describe how to generate these values.

With this in mind, we introduce the distribution parameter k . To generate a node n , bounded by its parent’s range $[L, U]$, we generate k uniformly random variables. The optimistic and pessimistic values of n are obtained by taking their maximum and minimum respectively:

$$v_1, \dots, v_k \sim \text{Uniform}[L, U]$$

$$\text{Opt}(n) = \max_i (v_i) \text{ and } \text{Pes}(n) = \min_i (v_i)$$

Furthermore, the original B* paper by Berliner [2] implemented that at least one descendant must have an optimistic value equal to that of the parent and one must have a pessimistic value equal to that of the parent. We refer to trees generated according to the above description as “**Berliner trees**”.

For experiments on PB*, Palay [8] used artificial game trees according to the same description with one modification. Whenever a newly expanded node has a range ≤ 2 , its optimistic and pessimistic values are set to the midpoint of that range. These trees are referred to as “**Palay trees**”.

2.2 Differently Structured Artificial Trees

Generating larger artificial game trees is not complicated: increasing the parameter values b and k leads to larger, more difficult-to-solve, game trees. However, all these trees have a very similar structure. For artificial trees according to the original formulation [2], the range of a node’s parent serves as a bound on the values the node may take. This imposes a consistently converging structure on the game tree. Different tree structures may instead allow some divergence to take place, as long as the tree still converges overall.

A new parameter is introduced to simulate differently structured trees: the growth factor g . The growth factor determines the limit on how much the values of nodes may diverge from their parent’s bounds. The values of a node with a parent $[L, U]$ are now bound by the range $[L - \Delta_g, U]$. Where Δ_g is defined as:

$$\Delta_g = (g - 1) \cdot (U - L)$$

To ensure these trees still converge, at least one child of each node should have a pessimistic value equal to or larger than the parent’s pessimistic value. Values generated for the children of a node can be adjusted to meet this requirement. Two styles of tree generation are proposed: *AdjustBefore* and *AdjustAfter*. Unlike “Berliner” and “Palay” trees, these styles do not force any children of a node to have its optimistic or pessimistic value. Both styles generate nodes such that at least one child’s bounds are always contained within the parent’s range $[L, U]$.

AdjustBefore. One child of every node is randomly selected to have its values generated as bound by $[L, U]$. Every other child is bound by $[L - \Delta_g, U]$ instead.

AdjustAfter. Initially, all nodes are generated bound by $[L - \Delta_g, U]$. Afterwards, it is checked whether at least one child is within $[L, U]$. If no such child is found, the values of a randomly selected child are re-generated to be bound by $[L, U]$.

Value Generation. *AdjustBefore* and *AdjustAfter* generate the values of nodes in the same way as described in Section 2.1. Only the bounds are altered:

$$v_1, \dots, v_k \sim \text{Uniform}[L - \Delta_g, U]$$

$$\text{Opt}(n) = \max_i (v_i) \text{ and } \text{Pes}(n) = \min_i (v_i)$$

Note that *AdjustBefore* and *AdjustAfter* behave identically whenever $g = 1$ because then $\Delta_g = 0$ and every node is generated as bound by $[L, U]$.

3 The B* Algorithm

3.1 General Formulation

Originally formulated in 1979, B* [2] is a best-first search algorithm for determining the best next move in a game tree. This report focuses on the 2-player formulation of B* as applied to adversarial game trees where nodes alternate by depth between maximizing (MAX) and minimizing (MIN) the score at the end of the game. Each node is given two evaluations: an optimistic and a pessimistic evaluation. These two evaluations provide a range of possible values for the game-theoretical value of the node's subtree. The maximum of this range is the optimistic value for MAX and the pessimistic value for MIN and vice versa. These bounds get tighter as the search goes on and new information about its subtree is backed up to the top. A node's bounds are updated based on the bounds of its children, as shown in formula 1 for MAX and formula 2 for MIN.

$$[Lower(x), Upper(x)] \rightarrow \left[\max_{c \in children(x)} Lower(c), \max_{c \in children(x)} Upper(c) \right] \quad (1)$$

$$[Lower(y), Upper(y)] \rightarrow \left[\min_{c \in children(y)} Lower(c), \min_{c \in children(y)} Upper(c) \right] \quad (2)$$

where x is a maximizing node and y is a minimizing node

This formulation allows a simple best-first algorithm to solve until a single value is obtained from the root. However, B* introduces two improvements that use the range-based heuristic to terminate the search quicker: node separation and the use of two search strategies at the root.

Termination at Separation. Node separation occurs when, among all children of a node, one child has a pessimistic value greater than or equal to the optimistic values of its siblings [2]. This means that the best child is proven to be the best move. Its sibling nodes no longer have to be considered. The search can terminate early when separation occurs at the root node: we know the proven best move and do not have to continue the search. Overall, B* selects a node, expands it, re-computes the bounds of every node above it, and repeats until separation occurs at the root.

Node Selection. Usually, B* [2] picks the child with the best optimistic value to be selected next. However, at the root, there are two strategies for the next node selection: Prove-Best and Disprove-Rest. Prove-Best selects the node with the best optimistic value, further exploring the most promising node. Disprove-Rest selects the node with the second-best optimistic value, attempting to disprove the alternatives to the most promising node. Both strategies increase the distance between the best node and its alternatives, eventually leading to separation. However, these strategies do not always perform equally well.

Strategy Selection. A simple example tree can display how the choice of strategy can affect the proof. Figure 1 shows a tree from [2] where Prove-Best and Disprove-Rest are both applicable. The current best move ① has a pessimistic value of 15, which is lower than the optimistic values of the alternatives ② 22 and ③ 19. There is no separation, so the search can not be terminated yet.

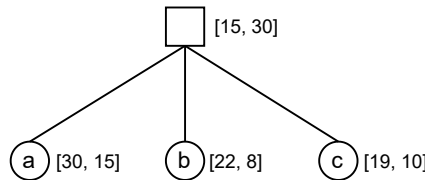


Fig. 1. Example tree from [2] with only the root node expanded. Nodes are displayed as squares (maximizing) or circles (minimizing) with their pessimistic and optimistic values in square brackets in that order. The best optimistic node ① is also the node with the best pessimistic value, so Prove-Best and Disprove-Rest are both applicable.

Figure 2 shows this tree solved using only the Prove-Best strategy. The search adjusts the bound of the best node ① until it is raised above the bounds of the alternatives ② and ③. Figure

3 instead shows the same tree solved using only the Disprove-Rest strategy. This requires fewer node expansions in this example: it is easier to disprove $\textcircled{b}$ and $\textcircled{c}$ than it is to prove $\textcircled{a}$. In this example, Disprove-Rest achieved a quicker proof than Prove-Best. Which strategy is best depends on the situation and may change every time the search returns at the root. For this reason, strategy selection is kept as a heuristic choice for the implementation.

However, Disprove-Rest may not always be applicable. B* [2] only allows Disprove-Rest to be selected when the node with the best optimistic value at the root also has the best pessimistic value. For example, let the root have children $\textcircled{x}$ [15, 5] and $\textcircled{y}$ [12, 8]. Disprove-Rest would select $\textcircled{y}$ which can never lead to separation. B* always selects the Prove-Best strategy in such cases.

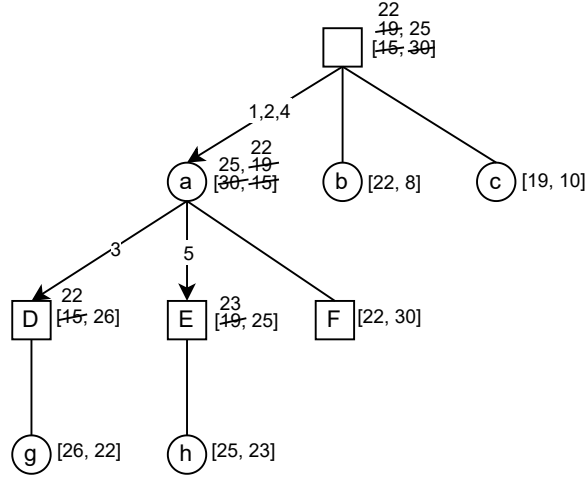


Fig. 2. Tree from Figure 1, by [2], solved using only the Prove-Best strategy. Numbers on edges show the order in which node selections were made. A node is expanded when it is selected for the first time. Updated range values are struck through and replaced above the old ones. The search terminates when $\textcircled{a}$ separates from $\textcircled{b}$ and $\textcircled{c}$.

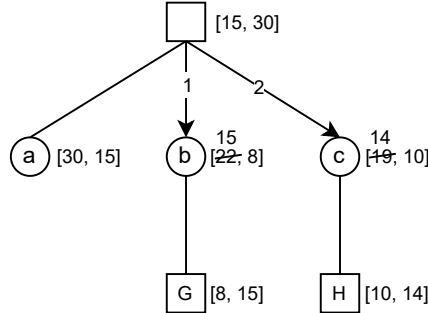


Fig. 3. Tree from Figure 1, by [2], solved using only the Disprove-Rest strategy. Instead of selecting the most optimistic node, Disprove-Rest expands the node with the second-best optimistic value. Compared with Figure 2, this example requires fewer iterations and expansions. Search still terminates due to separation of the best node $\textcircled{a}$.

3.2 Pseudocode

The B* algorithm [2] is expressed as pseudocode in Algorithm 1. Lines 1-3 expand the root node to initialize the search. Lines 4-9 encapsulate the main logic of the algorithm. Line 4 checks for the terminating condition of separation at the root. Recall that separation is reached whenever one child has a pessimistic value better than the optimistic value of all other children. Line 5 triggers

the sub-routine to select the strategy from Algorithm 2. Line 6 uses Algorithm 3 to select the next node to be expanded on line 7. After expanding the next node we have to back-propagate the information to the top. This is performed by the update Algorithm 4 on line 8. In Algorithm 4, line 18 updates the bounds of the node that was expanded on line 7. The update algorithm is recursively called on line 19 until we are backed up to a node where the bounds do not change. Algorithm 4 returns the first ancestor whose bounds do not change after updating, or the root node if they all do. B* thus does not always back up to the root and may back up to a lower node until bounds change all the way up to the root. Finally, in Algorithm 1, the best child is returned by the main B* solver on line 10. This returns the best move because the while loop on lines 4-9 continues until separation is reached and the best node is proven.

Algorithm 1 Solve(root) $\rightarrow$ bestMove

```

1: current  $\leftarrow$  root
2: expand(current)
3: computeAndSetBounds(current)
4: while not separation at root do
5:   if current = root then SelectStrategy(root)
6:   current  $\leftarrow$  SelectNext(current, root)
7:   expand(current)
8:   current  $\leftarrow$  Update(current)
9: end while
10: return best child node at root

```

Algorithm 2 SelectStrategy(root)

```

11: if ChildBestOpt(root)  $\neq$  ChildBestPes(root) then
12:   strategy  $\leftarrow$  ProveBest
13: else strategy  $\leftarrow$  ProveBest or DisproveRest  $\triangleright$  this is heuristically decided
14: end if

```

Algorithm 3 SelectNext(current, root) $\rightarrow$ next

```

15: if current = root and strategy is DisproveRest then
16:   return ChildSecondBestOpt(root)
17: else return ChildBestOpt(current)
18: end if

```

Algorithm 4 Update(node) $\rightarrow$ next

```

18: boundsChanged  $\leftarrow$  computeAndSetBounds(node)
19: if boundsChanged and node  $\neq$  root then
20:   return Update(parent of node)
21: else return node
22: end if

```

3.3 B* Strategy Selection

The previous section describes the B* algorithm [2] in detail. This algorithm requires a choice of Prove-Best or Disprove-Rest whenever both strategies are applicable at the root. The following section explains some methods of selecting the strategy for B* search.

Naive Approaches. The simplest way to choose a strategy is to default to one or the other. This is not a very effective method, as shown in Section 5, but a good baseline to compare to other methods. Two variants of B* [2] arise: **Always-PB** and **Prefer-DR**. Always-PB always selects Prove-Best. This is the default best-first search with only the separation stop condition added. Prefer-DR uses both Prove-Best and Disprove-Rest but always picks Disprove-Rest whenever it is applicable. This variant mostly explores the less promising nodes until separation is eventually achieved.

Neither variant makes good use of the benefits of having two strategies to choose from, so an alternative naive approach is to instead alternate between the two strategies: **B* Alternated**. This is the method applied by [11] for the endgame of Scrabble in the MAVEN program [10]. Note that we alternate between Prove-Best and Disprove-Rest only when the node with the best optimistic value is also the node with the best pessimistic value. This means that when there is one promising node over the others, we alternate between exploring the best and second-best node. A similar but non-deterministic method is the **Randomized** approach. This method randomly selects Prove-Best or Disprove-Rest when both are applicable. These two naive methods have the benefit of simplicity while still making use of the Prove-Best and Disprove-Rest strategies.

Heuristic Approaches. Along with the B* algorithm [2] itself, the author also described two heuristic strategy selection methods for its implementation. Both heuristics require the depth from which the optimistic bounds of all children at the root node have been backed up to be tracked. This value is referred to as d_{opt} . This has been implemented in this report as the depth from which the upper bound of the child’s range has been backed up. The **x D-criterium** chooses Disprove-Rest if the sum of d_{opt} squared of x alternatives is less than d_{opt} of the best child squared. This report implements this as the x alternatives at the root with the highest upper bound. Otherwise, the x D-criterium chooses Prove-Best. The alternative **x R-criterium** works the same as the x D-criterium but divides the squares of d_{opt} by $Range(n) = Upper(n) - Lower(n)$ for each child n of the root.

4 Disprove-Best for B*

We introduce an enhancement for the B* algorithm [2]: Disprove-Best, an alternative approach to lower-level node selection for quicker search. The following Section 4.1 gives the core inspiration for the algorithm. Next, Section 4.2 explains how Disprove-Best works. Section 4.3 goes into more detail with pseudocode. Finally, Section 4.4 gives a working example of node selection with Disprove-Best.

4.1 Inspiration

We were partially inspired to write Disprove-Best from the performance improvements achieved with PB* [8] even without a distribution-based evaluation function. In the paper, lower-level decisions in the tree allow for a search with fewer nodes expanded. The authors selected nodes to maximize the probability of raising or lowering a node’s bound above or below Δ . Here, Δ refers to the second-best optimistic value at the root, which we call b in this report. A similar principle is applied for Disprove-Best, but instead of working with probabilities, we look only at unit increases of the evaluation function.

We also took inspiration from what happens in B* [2] when the best optimistic node is not likely to be the best node overall. In this case, where it makes sense to “Disprove” the best node, we observed that Prove-Best does not always solve it efficiently. The Disprove-Best strategy seemed to work better in this scenario but is not meant to be applied in other cases. In summary, Disprove-Best takes inspiration from PB* [8], but is only used to solve cases where the best node can be disproven more efficiently.

4.2 The Disprove-Best Strategy

Disprove-Best is an alternative strategy applied where B* [2] would otherwise always choose the Prove-Best strategy. This is whenever the node with the best optimistic value at the root differs from the node with the best pessimistic value at the root. As with B*, the strategy is selected at the root and only re-evaluated when the search returns to the root. When Disprove-Best is chosen, two values are evaluated at the root: a , the best pessimistic value, and b , the second-best optimistic value. The search terminates if the range of the current best node updates to be above b , leading to separation. If however, its range updates to be below a , we can discard this node as it is disproven, and continue the search without it. The values a and b are only re-evaluated whenever Disprove-Best is selected again after the search returns to the root node.

Node Selection. Disprove-Best selects the node with the best optimistic value at the root. Contrary to B* [2], Disprove-Best does not necessarily select the node with the best optimistic value at lower levels. For each child n of the current node p two values are computed: the downward effort $e^\downarrow(n)$ and the upward effort $e^\uparrow(n)$. These are computed as shown in Formulas 3 and 4.

$$e^\downarrow(n) = Upper(n) - a \quad (3)$$

$$e^\uparrow(n) = b - Lower(n) \quad (4)$$

A comparison of the sum of upward efforts and the minimum downward effort determines the next selected node. The selected node is the node which represents the easier proof: pushing the range of p (A) downwards, below a , or (B) upwards, above b . For (A), this is the node with the minimum downward effort. For (B), this is the node with the maximum upward effort. If the current node p is maximizing instead of minimizing, the upward and downward efforts are swapped, and everything else remains the same.

We also introduce the effort ratio r as a parameter for the Disprove-Best strategy. The effort ratio is the expected ratio between the upward and downward efforts of a maximizing node. The sum of upward efforts is multiplied by r before being compared to the minimum downward effort. Which is the lowest determines the node selected, (A) for downward, (B) for upward. Figure 4 shows a diagram of the node selection process of Disprove-Best at a minimizing node with maximizing children nodes.

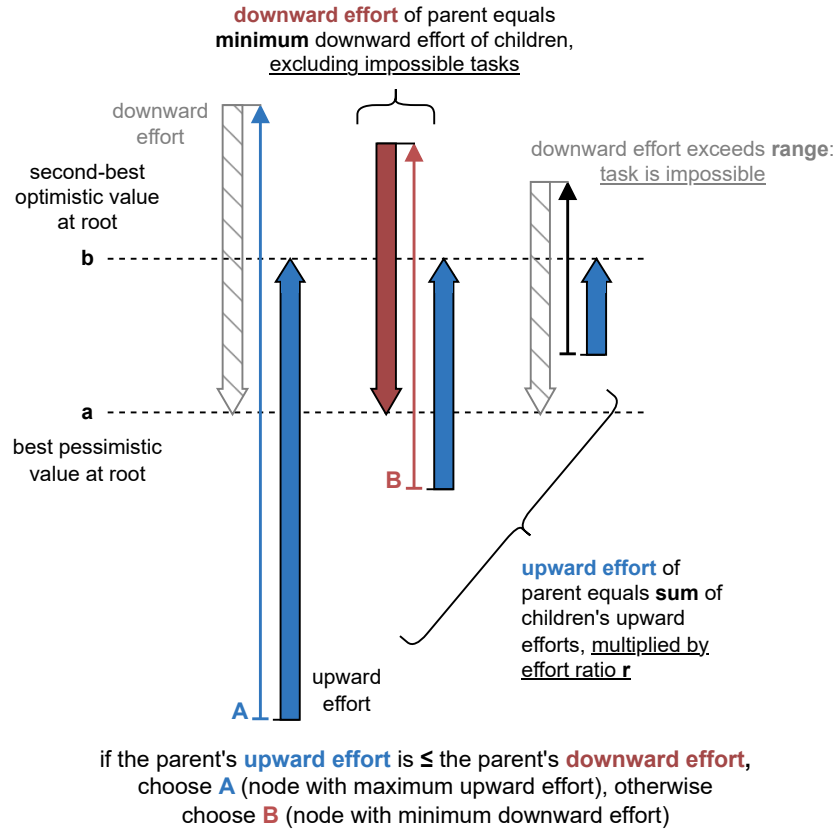


Fig. 4. Diagram showing visually the concepts of upward and downward effort in Disprove-Best. Thin arrows show children's ranges, with the direction showing that these nodes are maximizing. The parent is a minimizing node. The rightmost node has a downward effort greater than its range. This node is irrelevant to the downward effort of its parent, as it is impossible to push this node downwards below a .

4.3 pseudocode

To implement Disprove-Best we have to make some alterations to the B* algorithm [2] described in Section 3.2. First, we keep the core Algorithm 1 the same, but replace the strategy selection Algorithm 2 with Algorithm 5. In line 2, Disprove-Best is selected as the strategy, instead of Prove-Best as in the original description. Next, we replace the node selection Algorithm 3 with Algorithm 6. On line 7 we refer to the selection procedure specifically for Disprove-Best, which is given by Algorithm 7. The following section describes some important lines of pseudocode from this algorithm.

Disprove-Best Pseudocode. Lines 9 and 10 refer to how a and b are computed, but in implementation, a and b only have to be computed where Disprove-Best is selected as the strategy at the root. Lines 17-19 swap the upward and downward efforts if the child node is minimizing, which is the case when the parent node is maximizing.

Line 20 is an important condition: nodes with a zero or negative upward effort are irrelevant to the proof entirely. This is because these nodes already fulfil the requirement for pushing the parent node upwards, and can never make progress for pushing the parent downwards. Line 21 is a similarly important condition: nodes with a downward effort greater than their range are irrelevant for downward movement. Because node bounds can only get tighter when updated, we can never move a bound by more than its range. These nodes are still relevant for upward movement but are not considered for the minimum downward effort or best downward node.

Line 32 shows the exact condition used for determining the selected node. if $r \times \sum_n e^\uparrow(n) \leq \min_n e^\downarrow(n)$, we return the child with the maximum upward effort. Otherwise, the child with the minimum downward effort is returned.

Algorithm 5 SelectStrategy-DB(root)

```

1: if ChildBestOpt(root)  $\neq$  ChildBestPes(root) then
2:   strategy  $\leftarrow$  DisproveBest
3: else strategy  $\leftarrow$  ProveBest or DisproveRest ▷ this is heuristically decided
4: end if

```

Algorithm 6 SelectNext-DB(current, root)

```

5: if current = root and strategy is DisproveRest then
   return ChildSecondBestOpt(root)
6: else if current = root or strategy is not DisproveBest then
   return ChildBestOpt(current)
7: else
   return SelectWithDisproveBest(current, root)
8: end if

```

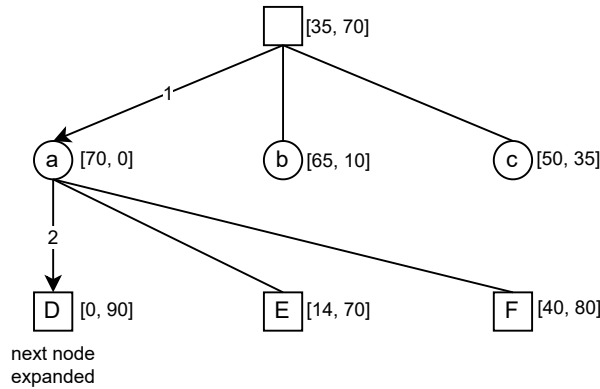
4.4 Disprove-Best Example

We construct a simple example tree which showcases the node selection obtained with Disprove-Best. It does not show how Disprove-Best might outshine the original B* implementation, as this is tested experimentally in Section 5. Figure 5 shows this example with the Prove-Best strategy. Note that the node $\textcircled{a}$ [70, 0] has the best optimistic value (70), but not the best pessimistic value (35). So Disprove-Rest is not applied in this example. Prove-Best selects the most optimistic node twice, returning $\boxed{D}$ as the next node to be expanded.

Disprove-Best. Figure 6 shows the same example with the Disprove-Best strategy, where $r = \frac{1}{3}$. All nodes shown in the figure have already been expanded previously, Disprove-Best only determines which node is expanded next here. We start by denoting the values a and b : the best pessimistic value, 35, and the second-best optimistic value, 65, respectively. Disprove-Best first selects $\textcircled{a}$, as

Algorithm 7 SelectWithDisproveBest(current, root, r) $\rightarrow$ next

Require: current $\neq$ root
 9: $a \leftarrow \text{BestPes}(\text{root})$
 10: $b \leftarrow \text{SecondBestOpt}(\text{root})$
 11: $\text{totalUp} \leftarrow 0$
 12: $\text{minDown} \leftarrow \infty$
 13: $\text{maxUp} \leftarrow 0$
 14: **for all** child of current **do**
 15: $\text{effortDown} \leftarrow \text{Upper}(\text{child}) - a$
 16: $\text{effortUp} \leftarrow b - \text{Lower}(\text{child})$
 17: **if** child is minimizing **then**
 18: $\text{effortDown}, \text{effortUp} \leftarrow \text{effortUp}, \text{effortDown}$
 19: **end if**
 20: **if** $\text{effortUp} > 0$ **then**
 21: **if** $\text{effortDown} < \text{minDown}$ **and** $\text{effortDown} \leq \text{Range}(\text{child})$ **then**
 22: $\text{bestDown} \leftarrow \text{child}$
 23: $\text{minDown} \leftarrow \text{effortDown}$
 24: **end if**
 25: **if** $\text{effortUp} > \text{maxUp}$ **then**
 26: $\text{bestUp} \leftarrow \text{child}$
 27: $\text{maxUp} \leftarrow \text{effortUp}$
 28: **end if**
 29: $\text{totalUp} \leftarrow \text{totalUp} + \text{effortUp}$
 30: **end if**
 31: **end for**
 32: **if** $r \times \text{totalUp} \leq \text{minDown}$ **then return** bestUp
 33: **else return** bestDown
 34: **end if**

**Fig. 5.** Example node selection with the Prove-Best strategy.

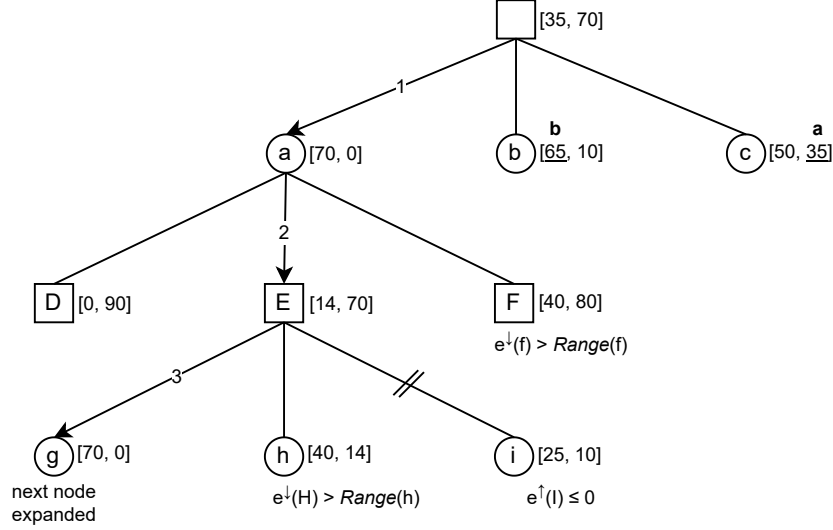


Fig. 6. Example node selection with the Disprove-Best strategy ($r = \frac{1}{3}$).

it is the best optimistic node at the root. Next, we have to calculate the upward and downward efforts of $\boxed{D}$, $\boxed{E}$, and $\boxed{F}$:

$$\begin{aligned}
 e^{\downarrow}(\boxed{D}) &= 90 - 35 = 55 & e^{\downarrow}(\boxed{E}) &= 70 - 35 = \mathbf{35} & e^{\downarrow}(\boxed{F}) &= 80 - 35 = 45 \\
 e^{\uparrow}(\boxed{D}) &= 65 - 0 = \mathbf{65} & e^{\uparrow}(\boxed{E}) &= 65 - 14 = 51 & e^{\uparrow}(\boxed{F}) &= 65 - 40 = 15
 \end{aligned}$$

Note that $\boxed{F}$ has a range of 40, so its downward effort of 45 makes pushing this node downwards impossible. Node $\boxed{F}$ is thus not considered for the minimum downward effort or best downward node. With $\boxed{F}$ excluded, the node with the minimum downward effort is $\boxed{E}$ at 35. As an alternative, the node with the maximum upward effort is $\boxed{D}$. The sum of all upward efforts is 131, times r , gives $\frac{1}{3} \times 131 = 43.\bar{6}$. Comparing these values shows that the lowest effort option is $\boxed{E}$: $35 < 43.\bar{6}$. Disprove-Best thus selects $\boxed{E}$ as the next node.

Determining the next node is similar, but take note that the parent node $\boxed{E}$ is maximizing, so the upward and downward efforts are swapped:

$$\begin{aligned}
 e^{\downarrow}(\textcircled{g}) &= 65 - 0 = \mathbf{65} & e^{\downarrow}(\textcircled{h}) &= 65 - 14 = 51 & e^{\downarrow}(\textcircled{i}) &= 65 - 10 = 55 \\
 e^{\uparrow}(\textcircled{g}) &= 70 - 35 = \mathbf{35} & e^{\uparrow}(\textcircled{h}) &= 40 - 35 = 5 & e^{\uparrow}(\textcircled{i}) &= 25 - 35 = -10
 \end{aligned}$$

We observe that $e^{\uparrow}(\textcircled{i}) \leq 0$, so $\textcircled{i}$ is irrelevant. Also notice that $e^{\downarrow}(\textcircled{h}) = 51 > \text{Range}(\textcircled{h}) = 26$, so $\textcircled{h}$ and $\textcircled{i}$ are both not applicable as best downward node. Therefore, the best downward node is also the node with the maximum upward effort: $\textcircled{g}$. Disprove-Best selects $\textcircled{g}$ to be expanded next, regardless of r .

5 Experiments

This section describes the experiments conducted to help answer the research questions raised in the introduction. The research questions are as follows:

1. How do existing implementations of the B* algorithm perform on larger artificial game trees?
2. How do the different parameters for tree generation affect their size and difficulty?
3. How does Disprove-Best B* perform compared to existing B* variants?

5.1 Experimental Setup

A singular large experiment was devised to test the performance of various algorithms on a variety of artificial game trees, accounting for all research questions. For this experiment, a run refers to the computation of the optimal next move from a specific game position. A game position is represented by an artificial game tree with some set parameters and a unique seed. As with the implementation by Berliner [2], the order in which a tree is explored does not affect its structure. For each unique tree, a run is performed for each algorithm. During this computation the following metrics are recorded: number of nodes expanded (N_x), number of nodes evaluated (N_v), maximum depth reached (d_{max}), elapsed execution time (t_δ), best move id (n_{best}), and the pessimistic value of the best move (v_{sep}).

Algorithms. This experiment was conducted with 16 different variants of several algorithms. These algorithms are illustrated in Table 1. The Disprove-Best algorithms require a choice for the expected upward-to-downward effort ratio r . The analytic value of r proved difficult to derive. A basic estimate was therefore used instead. To push the range of a maximizing node upwards, only one of its children has to be pushed downwards. Whereas to push the range downwards by some amount, all children have to be pushed upwards at least that amount. A simple estimate is therefore that it is approximately b times easier to push a node upwards, where the branching factor b is the average number of children in the tree. This gives an estimate of $r \approx v \cdot \frac{1}{b}$ where v is some value that may be tweaked to improve performance. In the experiments of this report, five values of v from 0.8 to 2.0 have been used.

Name	Shortened	How the strategy is selected at the root
Always-PB	BF	always Prove-Best (Best First)
Prefer-DR	DR	use Disprove-Rest where possible
Randomized	RND	selected randomly, no set seed
Alternated	ALT	alternates between Prove-Best and Disprove-Rest
x D-Criterium	B- x D	using x D-criterium (Section 3.3) where x is any of $\{2, 3, A\}$
x R-Criterium	B- x R	using x R-criterium (Section 3.3) where x is any of $\{2, 3, A\}$
Disprove-Best $\frac{v}{b}$ AR	DB-AR- v	using criterion-AR and Disprove-Best (Section 4) where $r = \frac{v}{b}$ and v is any of $\{0.8, 1.0, 1.4, 1.6, 2.0\}$
Disprove-Best $\frac{2}{b}$ ALT	DB-ALT	using alternated strategies and Disprove-Best (Section 4) where $r = \frac{2}{b}$

Table 1. List and descriptions of algorithms compared in the experiments. The variable b refers to the branching factor of the tree that is being solved.

Trees. The artificial game trees were initialized with all combinations of the following parameters:

1. Initial value range at the root R : $[1, 100]$, $[1, 800]$, $[1, 6400]$
2. Branching factor b : 3, 4, 5, 6, 7, 8, 9, 10
3. Number of alternate values k : 2, 3, 4, 5, 6, 7, 8, 9, 10
4. Growth factor g : 0.75, 1.0, 1.4, 2.0, 3.0
5. Tree-generation algorithm:
 - Berliner, Palay, AdjustBefore, AdjustAfter

The growth factor g only applies to AdjustBefore and AdjustAfter, whereas it is assigned a value of $g = 1$ for Berliner and Palay trees. For each combination of parameters, 153 trees with unique seeds were generated. Therefore, trees generated with AdjustBefore and AdjustAfter are over-represented in the data. This gives a total of 396,576 trees on which each variant algorithm was executed once. Thus a total of 6,345,216 runs were performed.

Intractable Runs. Because of the scope of this experiment, some limitations had to be imposed on the computation to restrict its execution time. Berliner [2] implemented a limit on the number of nodes evaluated and the maximum depth searched for each run. Due to advancements in hardware, these limits can be greatly relaxed in this experiment. Any computation exceeding 500,000 nodes evaluated ($N_v > 0.5 \cdot 10^6$) was marked as intractable and had its search terminated. This limit of 500,000 nodes is rather conservative, as much larger trees can easily be explored, but it was chosen because of the large number of runs computed in this experiment. Note that this limit is much looser than the 30,000 nodes limit in the original B* experiments [2]. A run marked as intractable in this way is terminated early, but not discarded. With the v_{sep} metric, the data collected from intractable runs can still be used. Recall that v_{sep} is the pessimistic value of the current best move (best node at the root). This corresponds to the minimum proven value the player-to-move achieves by playing that move. While this is not a perfect measurement of progress for the uncompleted computation of the best move, it is a decent estimate. The metric v_{sep} is used to compare the performance of algorithms on trees that could not be solved within the limit of 500,000 nodes evaluated.

Hardware. All runs were computed independently and in parallel on an AMD 5950X CPU with 64GB of available RAM. Because of the parallel execution of this experiment, the maximum number of evaluated nodes had to be set relatively low to keep memory usage within a reasonable range. All code for trees, algorithms, and the experiment itself was written and executed in Java 18.

5.2 Results

This section describes the analysis and visualization of the results from the experiments conducted as described previously.

Intractable Trees. Due to the limits of the simulation, some runs were not computed to completion and thus denoted as intractable. In total, 30.8% of trees are intractable to all tested algorithms. The initial range of values at the root R does not seem strongly correlated with the intractability of trees, whereas the branching factor b , growth factor g , and distribution parameter k all have a clear effect. This effect is displayed in Figure 7. It shows how the branching factor b and distribution parameter k reliably increase the number of intractable trees. Whereas the growth factor g gives the least intractable trees at $g = 0.75$ and the most at $g = 1.4$, and less again for $g \geq 2$. This figure also shows how trees generated with Berliner are slightly more intractable than AdjustBefore and AdjustAfter trees with growth factor $g = 1$. Palay trees are, as intended, less often intractable as well, although this pattern appears to be lessened where $k \geq 6$.

Algorithm Performance. The algorithms do not appear to be strictly better than one another on all possible trees. This is displayed in Figure 8, where the heat map depicts the fraction of all trees that a pair of algorithms had solved in common. Similar algorithms are visible as blocks of continuous colour in the graph. The most effective algorithm in terms of number of trees solved is DB-AR-2.0, which solved 97.7% of all trees solved by any algorithm, whereas Always-PB solved only 74.8%. Berliner-2D and Berliner-AR performed the best out of the B * variants originally proposed by [2] in terms of the number of nodes evaluated and the number of trees solved.

Across the experiment, the overhead in terms of execution time is largely overshadowed by the time and space requirements of node evaluation and expansion. For that reason, the metrics “nodes evaluated” and “nodes expanded” are the most important for algorithm performance. The most performing algorithms are then the algorithms that evaluate the fewest nodes to solve the most difficult problems.

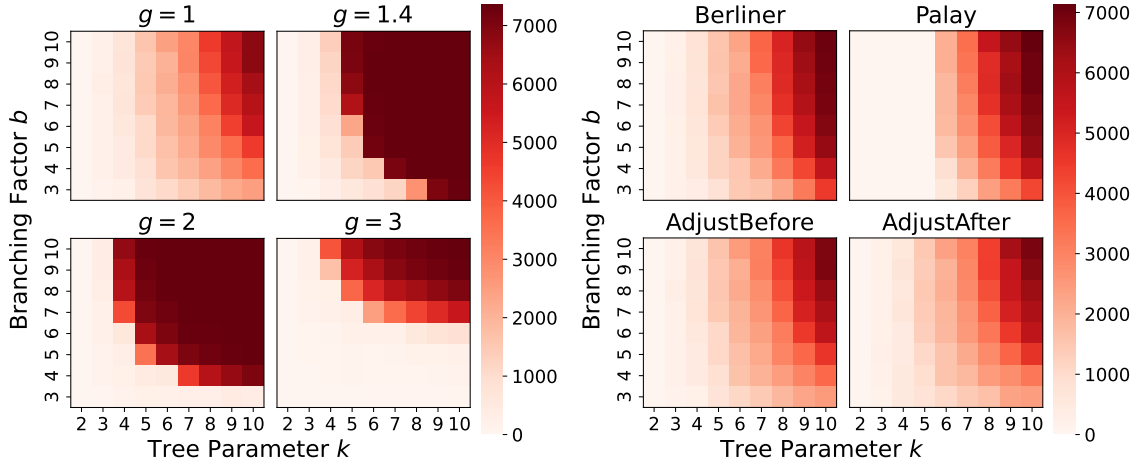


Fig. 7. Number of intractable trees by tree parameters b and k , and growth factor g (left) or Generation Style (right). On the (left), only trees with Generation Style AdjustBefore and AdjustAfter are included. On the (right) only trees with growth factor $g = 1$ are included.

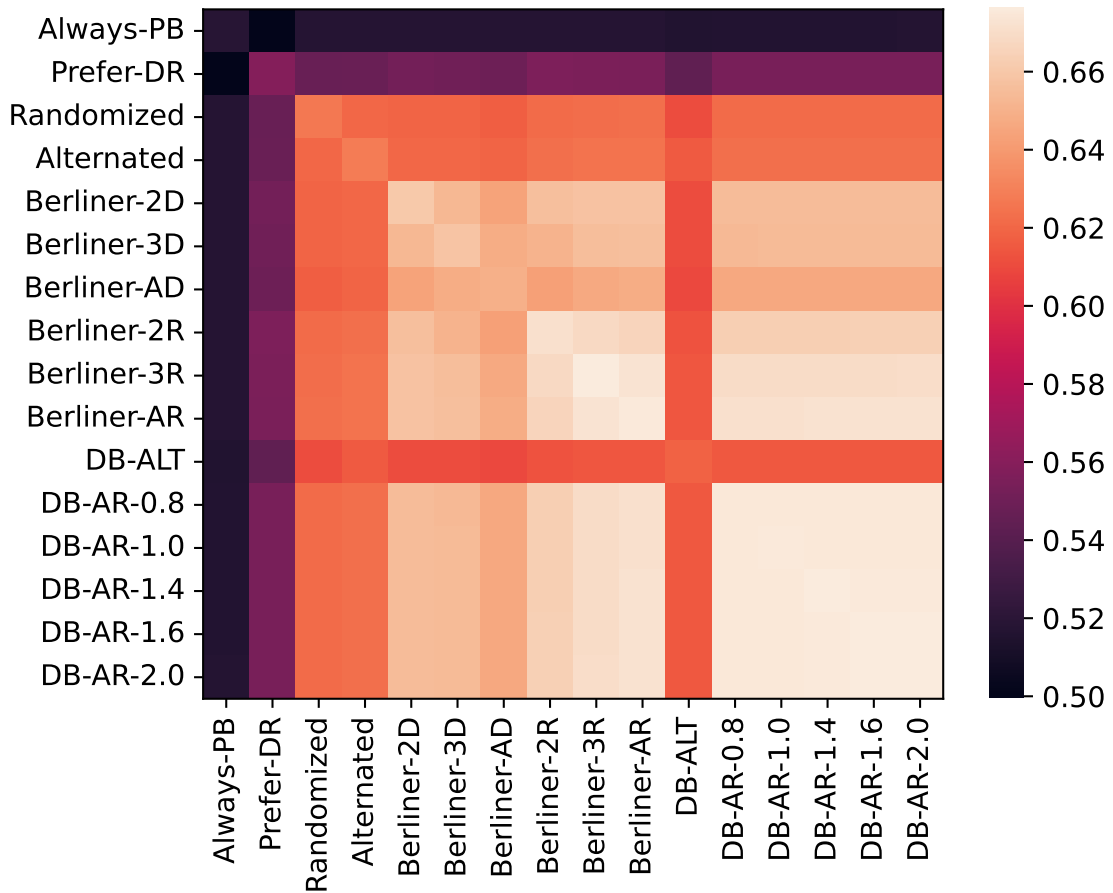


Fig. 8. Fraction of all trees solved in common by pairs of algorithms. The fraction of all trees solved by each algorithm can be read from the diagonal of this plot. A tree is considered solved by an algorithm if it is not intractable and, hence could be computed until separation with $\leq 500,000$ nodes evaluated.

A comparison of the number of nodes evaluated by six algorithms is shown in Figure 9. It shows that for smaller trees of < 5000 nodes, including those tested by [2] of < 1000 , DB-AR-2.0 and Berliner-AR perform the best and perform similarly. For larger trees, DB-AR-2.0 evaluates the least number of nodes, outperforming every other variant we tested.

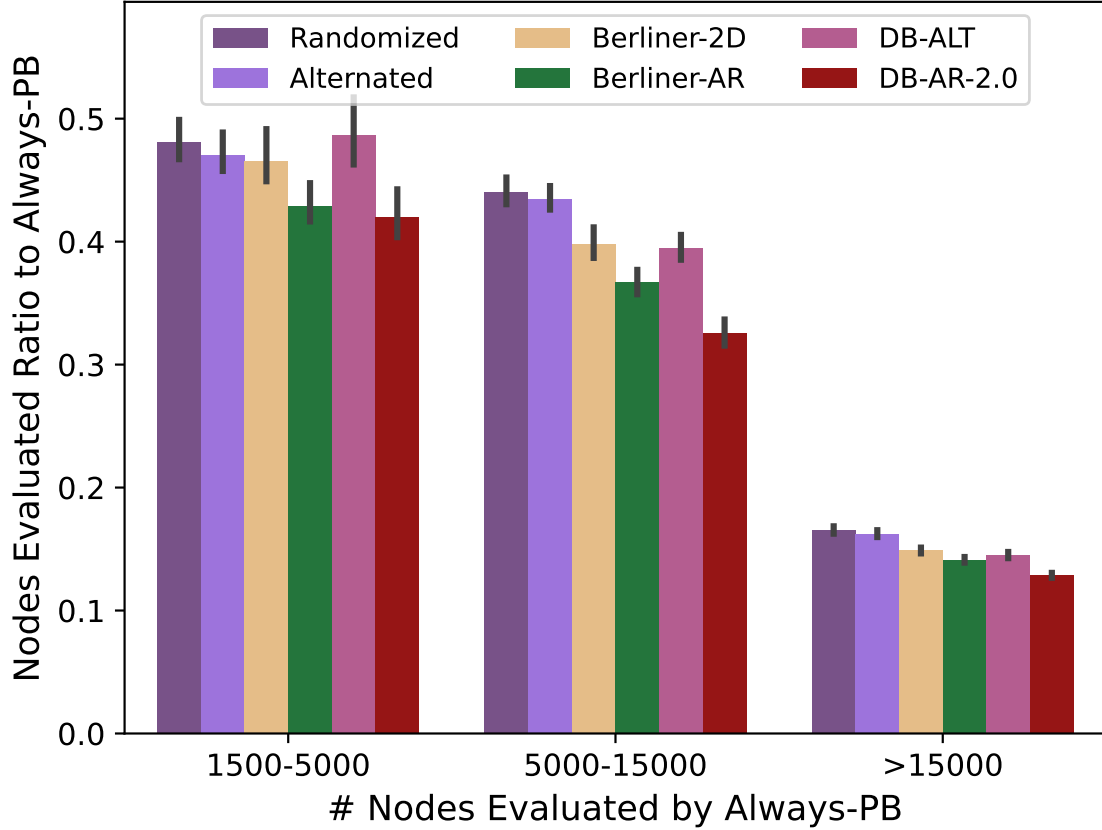


Fig. 9. Average number of nodes evaluated by six different algorithms divided by the number of nodes evaluated by Always-PB on the same tree. Lower is better. The averages are grouped by the number of nodes evaluated by Always-PB and include a 99%-confidence interval. Only trees for which all algorithms could find a solution by evaluating $\leq 500,000$ nodes are included in this graph.

Furthermore, Figure 10 shows the number of nodes expanded by four algorithms, by tree size, on large trees with growth factors $g = 0.75$ and $g = 1$. Note that tree ‘size’ refers to how many expansions were required to reach separation using the naive Always-PB implementation of B*. We see that trees with growth factor $g = 0.75$ require more node expansions from B* and Disprove-Best relative to Always-PB than trees with $g = 1$, but are most efficiently solved by Disprove-Best overall. Interestingly, $g = 1$ trees are more efficiently solved by B* in general, and even more as size increases, regardless of which variant is used. The DB-AR-2.0 variant performs better than all other variants tested for all tree sizes. However, accuracy for the largest trees is reduced due to a reduced number of samples for the rightmost data point in Figure 10.

Unsolved Trees. Figure 11 shows the performance of all algorithms on intractable trees, which account for 30.8% of the trees in this experiment. This figure shows the progress made by algorithms in terms of the v_{sep} metric. The value v_{sep} represents the proven minimum value obtained for the most optimistic move by the player-to-move at the end of the game. While the computation is incomplete, this metric encapsulates the proven advantage of the preliminary best move as computed by the algorithm so far. Note that all DB-AR- v algorithms performed very similarly and are therefore aggregated into a single bar in this figure. The figure shows that all Disprove-Best variants perform well, whereas Disprove-Rest performs the worst on this task.

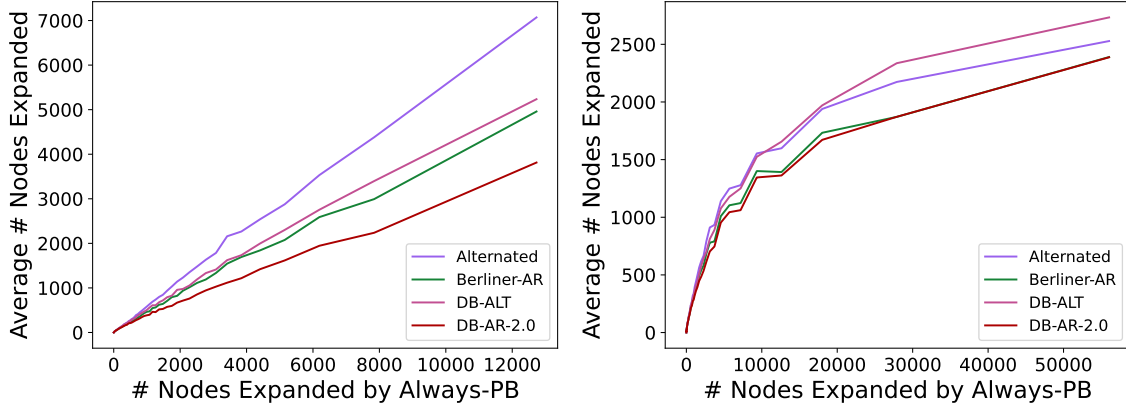


Fig. 10. Average number of nodes expanded by four different algorithms on trees with growth factor $g = 0.75$ (left) and $g = 1$ (right), relative to the number of nodes expanded by the Always-PB algorithm. Each point on the plot represents the average value of a fixed number of trees. A lower value means the algorithm solves the same problems with fewer resources (node expansions): performing better.

The left side of Figure 11 shows the progress made by algorithms on the largest trees. Always-PB obtained a v_{sep} of $< 0.2 \cdot R$ on these trees, as do most B* variants. Disprove-Best makes more than twice as much progress on these trees. This shows that Disprove-Best is good at advancing the pessimistic value of the best move at the beginning of the search. The worst-performing algorithm of prefer-DR is not surprising: this variant prefers not to expand the best node if possible.

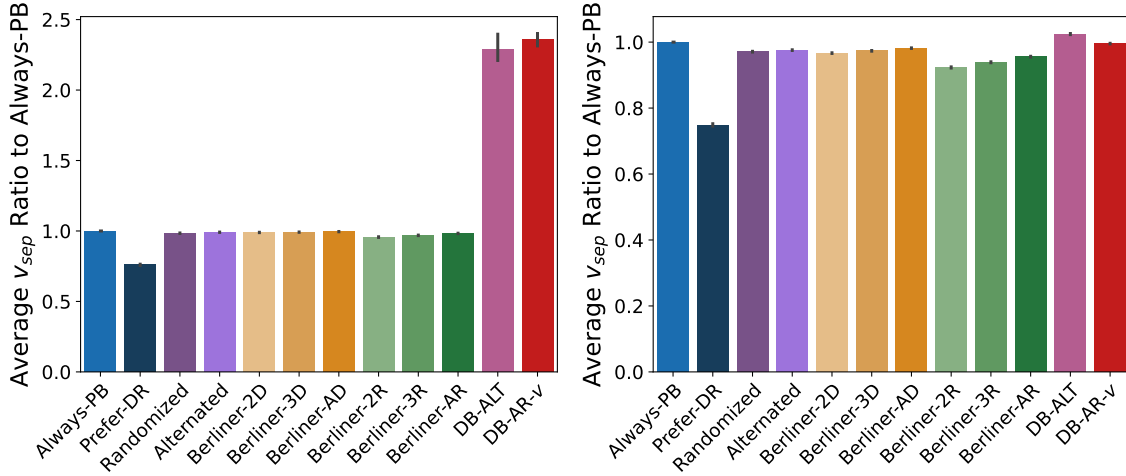


Fig. 11. Progress made on trees intractable for all algorithms, where $\frac{v_{sep}}{R}$ for Always-PB is < 0.2 (left) or ≥ 0.2 (right). Progress is estimated with the ratio of v_{sep} to the v_{sep} of Always-PB on the same tree. On the (left), trees on which Always-PB made very little progress are displayed, whereas the (right) displays trees where Always-PB made more progress. Higher is better. Each bar includes a 99.9%-confidence interval.

Interesting Patterns. Finally, v_{sep} can also be explored as a property of trees with different parameters. For solved trees, v_{sep} represents the minimum score the player-to-move can achieve by playing optimally. This can be referred to as a form of player-to-move advantage. Figure 12 shows a positive correlation between the branching factor b and v_{sep} , as well as a negative correlation between the growth factor g and v_{sep} . The tree parameter k also showed a weaker negative correlation with v_{sep} .

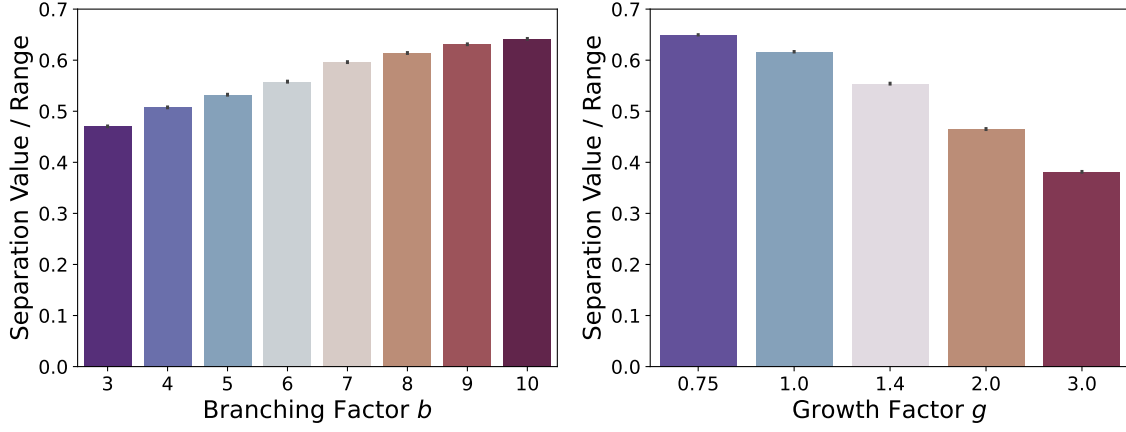


Fig. 12. Correlations between branching factor (left), growth factor (right), and the separation value $\frac{v_{sep}}{R}$ of fully solved trees. The separation value v_{sep} is the minimum score (on a range of 0 to 1, after dividing it by the initial range R) that the player-to-move can guarantee to achieve at the end of the game. Values are averages from trees solved by all algorithms. Each bar includes a 99.9%-confidence interval.

6 Comparison

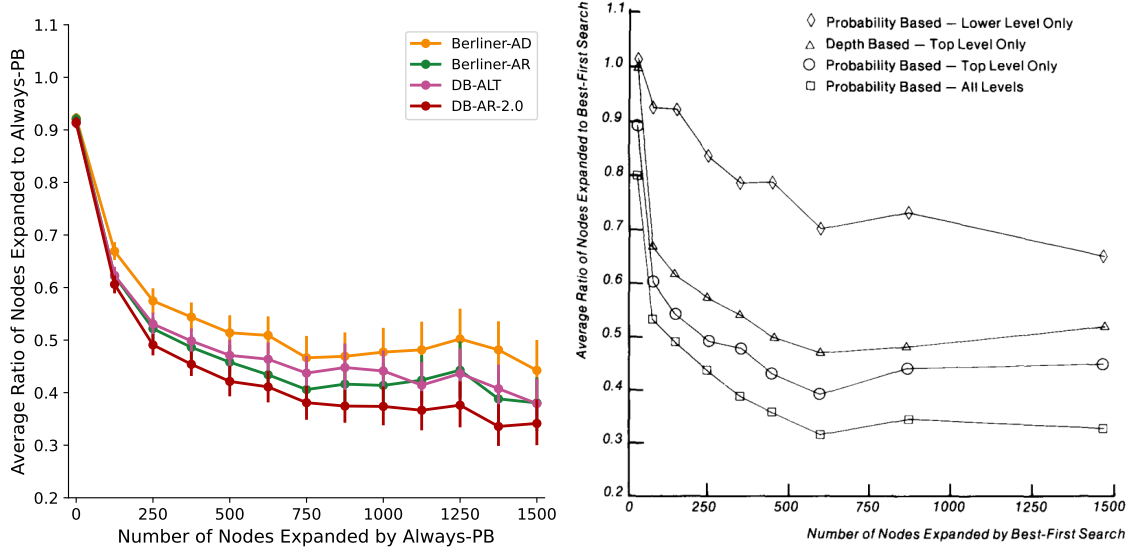


Fig. 13. Average number of nodes expanded by four different algorithms on Palay trees (left), divided by the number of nodes expanded by Always-PB on the same tree. Each point represents the average for a fixed range of tree sizes and has a 95%-confidence interval. The (right) shows the results of a similar experiment by Palay [8] on the PB* algorithm. The “Berliner-AD” algorithm (left) is identical to the “Depth Based” algorithm (right).

To evaluate how Disprove-Best performs compared to existing implementations we may also compare to existing research. PB* [8] expanded on the original B* implementation with a distribution-based evaluation function and altered node selection on all levels of the tree. Figure 13 shows the performance of four algorithms relative to Always-PB on “Palay” trees, generated according to the description of experiments by [8] on B* and PB*. The figure also shows the results from a similar experiment on the same trees by [8]. This allows for a direct comparison with the results from the PB* paper. The two graphs have one algorithm in common: Berliner-AD, referred to as “Depth Based - Top Level Only”, which is B* with the x D-criterion to determine the strategy at the top level of the tree. Both graphs in Figure 13 show similar results for Berliner-AD, which suggests the

other results may be comparable as well. This suggests that Disprove-Best with the AR-criterion (DB-AR-2.0) performs comparably to the “All Levels” variant of PB* in this experiment.

The results from [8] only describe trees of up to 1500 nodes and may not be representative of performance on larger trees. The novel experimental setup gives data for much larger trees as well. Figure 10 shows the results of a similar experiment but extended to much larger trees. These more difficult trees are more efficiently solved by B* and its variants than was previously observed on the smaller trees tested by [2] and [8]. This shows that B* performs better on larger trees than was previously observed. The same figure also shows that Disprove-Best performs better than B* in general. Disprove-Best is only compared to PB* on small trees in this report, where it performs similarly.

7 Conclusions and Future Research

In this report, the game-tree search algorithm B* [2] is revisited. The original experiments from the paper are reproduced on a larger variety of artificial game trees and compared to a novel B* variant: Disprove-Best. In this section, the results of this experiment are related to the research questions posed in the introduction.

7.1 Research Questions

How do existing implementations of B perform on larger trees?* It is observed (Figure 10) that the B*-algorithm performs more efficiently on game trees where the Always-PB approach requires more than 10,000 nodes to be expanded. These trees are between 10 and 40 times larger than those explored in the original B* experiments by [2] and [8]. Figures 9 and 10 show that, while Berliner-AR performs the best out of the B* variants, the much simpler Alternated B* variant is not much less effective on larger trees, relative to Always-PB (Best-First). This explains to some extent how Alternated B* remains relevant as a simpler implementation of B* that is game-agnostic. It is also shown (Figure 11) that the Alternated and Berliner- x D variants achieve more progress on intractable trees with 500,000 evaluated nodes than Berliner- x R does. Finally, Figure 8 shows that Berliner-AR is able to solve the largest fraction of trees within 500,000 evaluated nodes. This is only slightly less than the novel DB-AR- v algorithm, which makes use of the same x R heuristic.

How do the different parameters for tree generation affect their size and difficulty? Figure 7 shows the effect of tree parameters on the number of intractable trees, which is a measure of difficulty. The parameters b and k reliably increase the size of trees, whereas R and g do not have a linear relationship with tree size. The branching factor b increases the number of children that each node has, such that more nodes have to be explored to modify optimistic and pessimistic values at minimizing nodes. This results in larger, more difficult-to-solve trees. The distribution parameter k controls the average width of generated children bounds. A larger k leads to a slower convergence of optimistic and pessimistic values as depth increases. This also results in larger trees. The growth factor g increases difficulty conditionally on k and b , which is visible in Figure 7. Higher values of g allow for children nodes to have wider ranges than their parents, but ranges are not guaranteed to be wider. Some children nodes may be generated outside of parent bounds when g is large enough, making these nodes irrelevant to the search, which leads to smaller trees. An increase in g thus requires an increase in k as well such that more generated nodes can be wider than their parents, resulting in larger trees.

How does Disprove-Best B perform compared to existing implementations?*

Disprove-Best is an enhancement for B* which is inspired by the effectiveness of PB* [8] and the possibility of disproving the best move under certain conditions. It adds an alternate approach to node selection at lower levels of the tree: Disprove-Best. Disprove-Best is applied instead of Best-First whenever Disprove-Best is not a viable strategy at the root. This means it can be combined with other variants of B*, like Berliner- x D/- x R. The results show that the Disprove-Best algorithm DB-AR-2.0 solves the largest subset of trees (Figure 8), requires fewer node evaluations (Figure 9), requires fewer node expansions (Figures 10 and 13), and makes the most progress on unsolved trees with limited resources (Figure 11), as compared to basic B* variants. Disprove-Best with the AR-criterion (DB-AR-2.0) is shown (Figure 13) to perform similarly to PB* [8] without a distribution-based evaluation function.

7.2 Future Research

This report expands on the generation of artificial game trees for the comparison of range-based game-tree search algorithms. It also introduces a novel technique to improve the existing B* algorithm [2]: Disprove-Best. An interesting finding is that the efficiency of B* seems to increase with the difficulty of the tree. There are three lines of interesting future research. First, future research may look into this finding by comparing B* to other tree search algorithms on larger trees. Second, future research could implement the upper- and lower-level node selection of PB* [8] to the Disprove-Best approach. As Disprove-Best only applies under specific conditions, PB* could be used in the remaining cases to increase overall performance. Finally, future research may also explore possibilities to reduce the major shortcoming of the B* algorithm: memory allocation. While B* may perform more efficiently on larger trees, it is not practically applicable because a large number of nodes have to be kept in memory. Therefore, future research may experiment with existing or novel techniques to reduce memory usage for B* and related algorithms such as PB*.

References

1. Allis, L.V., Van der Meulen, M., Van den Herik, H.J.: Proof-number search. *Artificial Intelligence* **66**(1), 91–124 (1994). [https://doi.org/10.1016/0004-3702\(94\)90004-3](https://doi.org/10.1016/0004-3702(94)90004-3)
2. Berliner, H.J.: The B* tree search algorithm: A best-first proof procedure. *Artificial Intelligence* **12**(1), 23–40 (1979). [https://doi.org/10.1016/0004-3702\(79\)90003-1](https://doi.org/10.1016/0004-3702(79)90003-1)
3. Berliner, H.J., McConnell, C.: B* probability based search. *Artificial Intelligence* **86**(1), 97–156 (1996). [https://doi.org/10.1016/0004-3702\(95\)00092-5](https://doi.org/10.1016/0004-3702(95)00092-5)
4. Campbell, M., Hoane, A.J., Jr, Hsu, F.: Deep Blue. *Artificial Intelligence* **134**(1), 57–83 (2002). [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1)
5. Coulom, R.: Efficient selectivity and backup operators in monte-carlo tree search. In: *Computers and Games*. pp. 72–83. Springer Berlin Heidelberg, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-75538-8_7
6. Katz-Brown, J., O’Laughlin, J., Fultz, J.: Quackle 1.0.4. <https://github.com/quackle/quackle/releases/tag/v1.0.4> (2019)
7. Knuth, D.E., Moore, R.W.: An analysis of alpha-beta pruning. *Artificial Intelligence* **6**(4), 293–326 (1975). [https://doi.org/10.1016/0004-3702\(75\)90019-3](https://doi.org/10.1016/0004-3702(75)90019-3)
8. Palay, A.J.: The B* tree search algorithm—new results. *Artificial Intelligence* **19**(2), 145–163 (1982). [https://doi.org/10.1016/0004-3702\(82\)90034-0](https://doi.org/10.1016/0004-3702(82)90034-0)
9. Palay, A.J.: *Searching with probabilities*. Pitman Publishing, Inc. (1985)
10. Sheppard, B.: World-championship-caliber Scrabble. *Artificial Intelligence* **134**(1), 241–275 (2002). [https://doi.org/10.1016/S0004-3702\(01\)00166-7](https://doi.org/10.1016/S0004-3702(01)00166-7)
11. Sheppard, B.: Endgame play in Scrabble. *ICGA* **26**(3), 147–164 (2003). <https://doi.org/10.3233/ICG-2003-26302>
12. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., Hassabis, D.: Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016). <https://doi.org/10.1038/nature16961>

A First-Person Variant Disprove-Best

The pseudocode in Algorithm 8 depicts how Disprove-Best may be implemented for single-player games, with non-adversarial trees. Note that this variant has not been tested and is only a concept following the same principles as the Disprove-Best algorithm for adversarial trees.

Algorithm 8 SelectWithDisproveBest(current, root, r) $\rightarrow$ next

Require: current $\neq$ root $\triangleright$ use this version for non-adversarial trees
 $a \leftarrow \text{BestPes}(\text{root})$
 $b \leftarrow \text{SecondBestOpt}(\text{root})$
totalDown $\leftarrow 0$
minUp $\leftarrow \infty$
maxDown $\leftarrow 0$
for all child of current **do**
 effortDown $\leftarrow \text{Upper}(\text{child}) - a$
 effortUp $\leftarrow b - \text{Lower}(\text{child})$
 if effortDown > 0 **then**
 if effortUp $< \text{minUp}$ **and** effortUp $\leq \text{Range}(\text{child})$ **then**
 bestUp $\leftarrow \text{child}$
 minUp $\leftarrow \text{effortUp}$
 end if
 if effortDown $> \text{maxDown}$ **then**
 bestDown $\leftarrow \text{child}$
 maxDown $\leftarrow \text{effortDown}$
 end if
 totalDown $\leftarrow \text{totalDown} + \text{effortDown}$
 end if
end for
if totalDown $\leq r \times \text{minUp}$ **then return** bestDown
else return bestUp
end if
